

Forms & Inputs

Create Accessible, Usable Web Forms

■ FORMS

■ LEARNING OUTCOME

By the end of this module you will build fully accessible HTML forms with every input type, implement built-in HTML5 validation, style forms with CSS, and understand the accessibility requirements that make forms usable for everyone.

■ Skills You Will Gain

- Use all HTML5 input types with appropriate attributes
- Associate labels with inputs for accessibility
- Implement HTML5 built-in validation: required, pattern, min/max
- Group related inputs with fieldset and legend
- Style form inputs with CSS pseudo-classes
- Create custom select, checkbox, and radio button styles

■ Forms are where users convert — sign up, check out, contact, submit. A well-designed form increases conversions by 40%. A poorly designed form is the #1 reason users abandon a site.

SECTION 1

Form Element and Attributes

```
<form action="/submit" <!-- Where to send data --> method="POST" <!-- POST: data in body, GET: data in URL --> enctype="multipart/form-data" <!-- Required for file uploads --> novalidate <!-- Disable browser validation (use JS instead) --> autocomplete="on" <!-- Allow browser to autofill --> > <!-- form content --> </form> <!-- GET vs POST: GET: ?name=Alice&email=alice@example.com visible in URL Used for: search, filters, bookmarkable queries POST: data in request body, not visible in URL Used for: login, registration, payments, anything private -->
```

SECTION 2

Input Types and Attributes

Input Type	Behaviour & Mobile Keyboard
text	Plain text. Mobile: default keyboard
email	Email format validation. Mobile: shows @ key, .com button
password	Masked characters. Add autocomplete='current-password'
number	Numeric input with up/down arrows. Mobile: numpad
tel	Phone number. Mobile: shows numpad (no format validation)
url	URL format. Mobile: shows .com and / keys
search	Styled as search box with clear button
date	Date picker — format varies by browser
datetime-local	Date + time picker
time	Time picker
range	Slider between min and max values
color	Color picker
file	File picker — add accept='.jpg,.pdf' to filter
checkbox	True/false toggle — group for multiple options
radio	Single selection from group — use same name attribute
hidden	Not shown — sends data with form (CSRF tokens, IDs)

SECTION 3

Accessible Form Structure

```
<form action="/register" method="POST"> <!-- Fieldset groups related fields semantically --> <fieldset> <legend>Personal Information</legend> <!-- ALWAYS associate label with input using for + id matching --> <div class="form-group"> <label for="full-name">Full Name <span aria-hidden="true">*</span></label> <input type="text" id="full-name" name="fullName" required minlength="2" maxlength="50" autocomplete="name" aria-required="true" aria-describedby="name-hint"
```

```
placeholder="Jane Doe" > <span id="name-hint" class="hint">Enter your first and last name</span>
</div> <div class="form-group"> <label for="email">Email Address <span
aria-hidden="true">*</span></label> <input type="email" id="email" name="email" required
autocomplete="email" placeholder="jane@example.com" aria-invalid="false" <!-- change to true on
error --> > </div> <div class="form-group"> <label for="phone">Phone Number</label> <input
type="tel" id="phone" name="phone" pattern="[0-9]{10}" placeholder="9876543210"
aria-describedby="phone-format" > <span id="phone-format" class="hint">10-digit number, no
spaces</span> </div> </fieldset> <fieldset> <legend>Preferences</legend> <!-- Checkbox group -->
<div class="form-group" role="group" aria-labelledby="skills-label"> <span id="skills-label">Skills
(select all that apply)</span> <label class="checkbox-label"> <input type="checkbox" name="skills"
value="html"> HTML </label> <label class="checkbox-label"> <input type="checkbox" name="skills"
value="css"> CSS </label> <label class="checkbox-label"> <input type="checkbox" name="skills"
value="js"> JavaScript </label> </div> <!-- Radio group --> <fieldset> <legend>Experience
Level</legend> <label><input type="radio" name="level" value="beginner" required> Beginner</label>
<label><input type="radio" name="level" value="intermediate"> Intermediate</label> <label><input
type="radio" name="level" value="advanced"> Advanced</label> </fieldset> <!-- Select dropdown -->
<div class="form-group"> <label for="country">Country</label> <select id="country" name="country"
autocomplete="country-name"> <option value="">Select a country...</option> <optgroup label="South
Asia"> <option value="IN">India</option> <option value="PK">Pakistan</option> </optgroup> </select>
</div> <!-- Textarea --> <div class="form-group"> <label for="bio">About You</label> <textarea
id="bio" name="bio" rows="4" maxlength="500" placeholder="Tell us about yourself..."></textarea>
<span class="hint"><span id="char-count">0</span>/500</span> </div> </fieldset> <button
type="submit">Create Account</button> <button type="reset">Clear Form</button> </form>
```

SECTION 4

Styling Forms with CSS

```
/* ===== FORM BASE STYLES ===== */ .form-group { display: flex; flex-direction: column; gap: 6px;
margin-bottom: 20px; } label { font-weight: 600; font-size: 0.9rem; color: #374151; } /* Style all
text-like inputs consistently */ input[type="text"], input[type="email"], input[type="password"],
input[type="tel"], input[type="number"], input[type="url"], textarea, select { width: 100%;
padding: 10px 14px; border: 2px solid #d1d5db; border-radius: 6px; font-size: 1rem; font-family:
inherit; /* IMPORTANT: inputs don't inherit font */ color: #1c1917; background: white; transition:
border-color .2s, box-shadow .2s; } /* Focus state – NEVER remove, accessibility requirement */
input:focus, textarea:focus, select:focus { border-color: #e44d26; outline: none; box-shadow: 0 0 0
3px rgba(228, 77, 38, 0.2); } /* Valid and invalid states */ input:valid:not(:placeholder-shown) {
border-color: #10b981; } input:invalid:not(:placeholder-shown) { border-color: #ef4444; } /* Error
message styling */ .field-error { color: #ef4444; font-size: 0.85rem; } /* Disabled state */
input:disabled, select:disabled { background: #f3f4f6; color: #9ca3af; cursor: not-allowed; } /*
Submit button */ button[type="submit"] { padding: 12px 32px; background: #e44d26; color: white;
border: none; border-radius: 6px; font-size: 1rem; font-weight: 600; cursor: pointer; transition:
background .2s, transform .1s; } button[type="submit"]:hover { background: #c43e1c; }
button[type="submit"]:active { transform: scale(0.98); } button[type="submit"]:focus { outline: 2px
solid #e44d26; outline-offset: 2px; }
```

■ PRACTICE Q & A

Q1. Why must every input have an associated label?

A: *Accessibility:* screen readers announce the label when an input receives focus. *Usability:* clicking the label also focuses the input (larger click target). *SEO:* improves form field understanding. Associate with matching `for='id'` and `id` attributes.

Q2. What is the difference between placeholder and label?

A: *Label* is permanent and always visible — it describes the field. *Placeholder* disappears when the user starts typing — it's a hint or example value, NOT a label replacement. Many users forget what a field requires once the placeholder disappears.

Q3. What does the required attribute do vs HTML5 pattern validation?

A: *required:* field must have a value before form submits. *pattern:* value must match the regex pattern. Both are client-side only — ALWAYS validate on the server too, as client-side validation can be bypassed.

Q4. What is the `enctype='multipart/form-data'` attribute for?

A: Required when the form contains file inputs. The default `enctype='application/x-www-form-urlencoded'` cannot transmit binary file data. `multipart/form-data` packages each field separately, including file contents.

Q5. Why should radio buttons of a group share the same name attribute?

A: The name attribute groups radio buttons. Only ONE radio button with the same name can be selected at a time. Without matching names, all radio buttons become independently selectable like checkboxes.

■ Forms & Inputs Cheat Sheet	
<code>label for='id'</code>	Link label to input – required
<code>input type='email'</code>	Email with built-in validation
<code>input required</code>	Must fill before submitting
<code>input minlength='2'</code>	Minimum character count
<code>input pattern='regex'</code>	Custom regex validation
<code>input autocomplete</code>	Help browser autofill correctly
<code>fieldset + legend</code>	Group and label related inputs
<code>aria-describedby</code>	Link input to hint/error text
<code>aria-invalid='true'</code>	Mark field as invalid for screen readers
<code>input:focus { outline }</code>	NEVER remove focus style
<code>input:valid</code>	Style when value passes validation
<code>font-family: inherit</code>	Inputs don't inherit fonts – set it